2025 IEEE/ACM 37th International Conference on Software Engineering Education and Training (CSEE&T) | 979-8-3315-3709-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/CSEET66350.2025.00044

Bug Hunting Games to Add Enthusiasm in Software Testing and Programming Classes

Natalia Silvis-Cividjian *, Jasper Veltman *, Auke Buchel *, Erik Link *, Joshua Kenyon * and Michel Oey †
* Department of Computer Science, Vrije Universiteit (VU), Amsterdam, The Netherlands
† Faculty of Digital Media and Creative Industries, Amsterdam University of Applied Science and MakeIT Work, The Netherlands

***Abstract*—Although high-quality software is key for a safe society, beginners still struggle to understand programming basics, whereas experienced programmers consider testing code as an unnecessary, boring chore. The situation will improve when, instead of asking students to write hypothetical code or test cases, one challenges them to find and fix bugs deliberately injected in real, executable code. Since 2020, we have been developing educational artifacts needed to test this radically new hypothesis, including a web-based bug-hunting game for standalone code, and small-scale cyber-physical systems (a self-driving car and a smart home), controlled by fault-seeded embedded software. We deployed the proposed approach and artifacts in the following settings: a software testing course for 300 computer science students, an exam for 250 economics students enrolled in an introductory Python course, and a software testing refresher workshop for 80 alumni of an IT retraining programme. Evaluations based on surveys with Likert-scale and open questions showed that a fault-based, active-learning approach (1) increases excitement in learning and (2) offers a more adequate way to assess students' skills, compared to the traditional pen-and-paper or MC exams. Future work includes equalizing the difficulty level of injected bugs and adding other gaming elements, such as badges, scoreboards, automated grading and adaptive feedback. While we realize that more rigorous qualitative and quantitative evaluations will be needed to confirm the generalizability of our approach, we are confident in its potential to contribute to making future professionals better prepared to engineer the high-quality, safe software we all can rely on.**

***Index Terms*—teaching software testing, teaching introductory programming, fault-injection, testing embedded software**

I. Introduction

Although our society relies increasingly on software, it is still challenging to effectively teach novices how to engineer high-quality products. Two problems are in particular persistent.

#1. The first steps in learning programming are tough.

For many computer science (CS) and especially non-CS students enrolled in an introductory programming course, it takes a long time until the ball is rolling. When this never happens, ultimately students will drop out [1], [2], [3]. Furthermore, in many institutions the number of students enrolled in introductory programming courses is huge, making one-to-one coaching and providing timely personalized feedback practically impossible. In this situation, many students under stress are tempted to ask help from outsiders, or use generative AI tools like ChatGPT, creating frustration for teachers and seldom providing long-lasting success for students themselves.

#2. Students lack the motivation to learn software testing.

Testing is an activity that checks whether a software product works as expected [4]. Because there is no time to test everything, a test plan is needed containing selected test cases. The problem is that CS students tend to be overconfident that their software is the best and does not need to be tested. Therefore, they often submit low-quality, or AI-generated test plans. Moreover, it is known that students find testing unattractive when compared to design or coding [5], [6], [7], [8]. And we must admit that testing *is* indeed a rather boring activity, being just a repetitive cycle of making tests, feeding them to the software under test and judging whether a test case passed or failed, followed by a huge test report. The general adversary effect of this situation is the lack of well-prepared and motivated software engineers on the job market and a skill gap between industry and academia [9], [10], [11], [6], [12], leading to poor quality software in many sectors, and in extreme cases losses of data, assets, brand image, and even lives.

We are a team of software testing and programming educators who have been struggling with these problems for many years. In our opinion, the reason for this situation is twofold. On the one hand, traditionally (1) students are asked to write code for rather dull requirements (like array sorting, or the triangle classification problem) that contrast with the exciting, smart world they are living in, and, on the other hand, (2) they are asked to test hypothetical software, or in the best case, existing software that is written by experienced and well-intentioned programmers featuring few-to-zero bugs. In our quest to alleviate this situation, we came across the concept of gamification, defined as the strategy that uses the highly engaging character of games for other, non-ludic purposes [13], [14]. Inspired by the recent success of gamification in increasing motivation and fun in education, we decided to experiment with this new concept combined with the use of negative examples, mistakes and bugs [15]. We followed the Papert's constructionism learning paradigm [16], based on the belief that learning occurs when learners are actively involved in a process of knowledge construction. The mission statement of this initiative can be formulated as follows:

To create an authentic learning experience by engaging students in bug-hunting games that involve real executable code which appeals to their imagination.

In order to assess the feasibility of our idea, we first developed an educational infrastructure consisting of a web-

2832-7578/25/$31.00 ©2025 IEEE
DOI 10.1109/CSEET66350.2025.00044

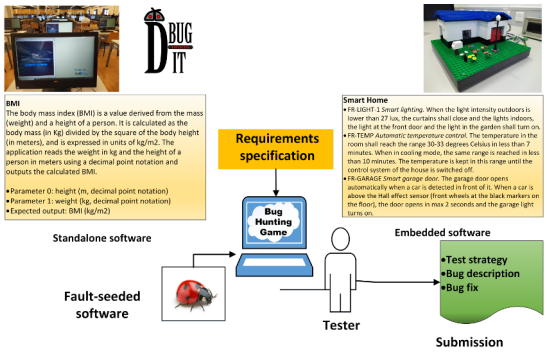


Fig. 1. The principle of the bug-hunting games.

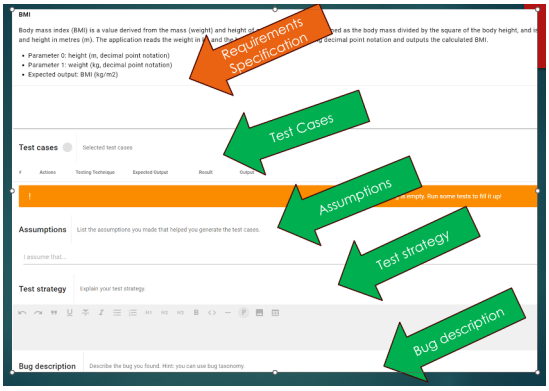


Fig. 2. DBugIT, a bug-hunting online game.

based bug-hunting game and microcontroller-based miniature "toys" that mimic modern real-life systems, such as a self-driving car and a smart home. All educational artifacts are very versatile and can be used in different teaching scenarios. They all work based on the same principle - students are provided with the requirements specification of an executable software unit, and must design a test strategy, find the hidden bugs and report on their findings. The teachers can grade the submissions off-line (see Fig. 1).

Next, we took steps to evaluate the feasibility of the proposed approach. We deployed all artifacts to a CS software testing course at the Vrije Universiteit in Amsterdam. Second, we transferred the teaching concept to an introductory Python programming exam taken by economics and business students from the same university. Finally, we used the web-based bug-hunting game in a software testing workshop for alumni of an IT retraining course at the Amsterdam University of Applied Sciences.

We evaluated the interventions by means of students' surveys and self-reports, with the aim of answering the following research questions.

RQ1. How does playing a bug-hunting game affect the excitement level of learning software testing?

RQ2. Is immersion in a bug-hunting experience a better way to assess software engineering skills compared to traditional exams?

RQ3. What emotions are triggered during a bug-hunting game?

The results so far are promising. Although we are aware that more systematic research will be needed to validate and possibly generalize the proposed approach, we believe in its potential to increase enthusiasm in introductory classes and offer a more robust way for assessment. In the following sections, we will share details of the infrastructure design and the results of some preliminary experiments.

II. The Design of the Educational Infrastructure

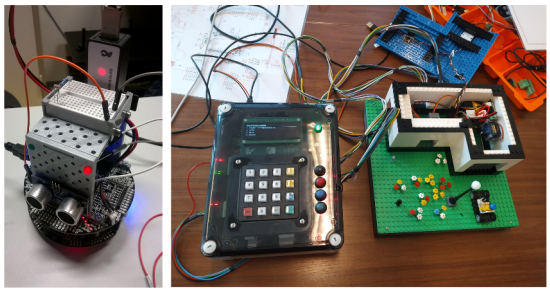
A. A web-based bug-hunting game

DBugIT is a product of the VU-BugZoo project [17], started in 2019 with the aim of making learning software testing more exciting. It is a novel interactive web application, in which students can "play" a simple detective game with standalone code that was deliberately contaminated with a bug (also called a mutant) [18]. Examples are a control software for an automatic insulin pump, or a discount calculator for web-shop clients. Examples of bugs we injected are commonly made programming errors, such as: off-by-one faults, an excluded or an extra boundary of a domain, too many or too few outputs, or an omitted check for division by zero. Students are provided with the requirements specification and can feed different test inputs to the executable (see Fig. 2).

The code can be made visible or not, depending on the wishes of the teacher. The executable is run on a remote server and students can immediately see the output of their test cases. DBugIT provides an advanced editor, where students can type in anything they want to submit. The database stores and manages the student and teacher related data - from exercises metadata to administration related data. The back-end functionality was developed using Typescript with Express for the APIs and MongoDB with Mongoose for the database. The executable code runs in Docker containers to prevent crashes due to improper input or division by zero. The front-end functionality was implemented using Vue.js [19], an open-source JavaScript framework for building fast and robust user interfaces. The tool allows hundreds of students to work online at the same time. The application allows creating separate user communities, with different roles, such as student or teacher with administrator's rights. Currently, the exercises database is populated with 70 exercises in Python and C++, and each teacher can easily assign exercises to their students. New exercises can be also added to the database, but in a less straightforward way at this moment.

B. Small-scale cyber-physical systems

In order to expose students to appealing software, we extended the educational infrastructure with two types of small-scale cyber-physical systems that mimic a real-life self-driving car and a smart home. The main motivation for this approach was existing evidence that in general, bringing to



a) b)

Fig. 3. Educational infrastructure for embedded software. a) An autonomous car. b) A smart home and its control unit.

class concrete every-day objects, also known as *realia* or physical manipulatives, can enhance learning [20].

The first system is a miniature autonomous car (see Fig. 3 a)) that has at its heart a commercially available Pololu m3pi robot [21]. On top of it an Arduino microcontroller is mounted, hosting C++ code. The car is equipped with two servomotors connected to the wheels and an array of five active optosensors that can measure, for example, the colour of a track on the floor needed to implement a stay-in-lane feature. We augmented the original system with a 3D-printed body, including a small breadboard on the roof. Innovative is that the construction of the car is not finished; students still have to do some wiring on the breadboard for example to connect a passive optical sensor to implement the adaptive lights feature. Other sensors can be easily attached. For example, we extended the sensory range with an ultrasonic sensor to measure the distance and facilitate the implementation of a collision detection feature.

The second ”toy” system is a smart home (see Fig. 3b)), which is also a product of the VU-BugZoo project. The home combines Lego blocks with a variety of sensors (light, temperature, Hall effect) and actuators (motors for curtains and doors, a heater and a ventilator, indoor and outdoor lights) [22]. The smart home’s electronics is connected to a multifunctional control unit, developed in-house based on an NXP-FRDM-K66F microcontroller [23], hosting the C++ software-under-test. The control unit front panel features I/O analog and digital ports, LEDs and push buttons, a buzzer, an alphanumerical display and a keypad, that can be used to communicate with the user and implement for example, a door access feature. The smart home is ”student-proof”; all the critical electronic components are adequately protected against damage caused by overheating or wrong electrical connections.

III. Preliminary Experiments

A. A software testing course for CS students

In May 2020 we used DBugIT for the first time to remotely assess the unit testing skills of 50 students enrolled in an introductory software testing course during the Covid-19 lockdown. Each student had to solve one black-box and one white-box testing exercise. For the given software-under-test, students were asked (1) to design an adequate test plan, including a strategy with assumptions, suggested test techniques and the associated test cases, and (2) to describe as accurately as possible the bug(s) they found using the Beizer bug taxonomy [24]. In the grading scheme, the soundness of the test strategy had the highest weight (50%), followed by the quality of test cases (25%) and the description of the bug (25%). We deployed the same approach in the next three years. In May 2024, due to the upcoming ChatGPT dangers, we switched from online to a proctored exam on campus. In total, we examined approximately 300 students. Every year, we evaluated the intervention using anonymous students’ surveys and asked the three research questions.

TABLE I
Excerpt from CS students’ evaluations of DBugIT.

Question	-	+-	+	Total	Agree
Testing is more exciting	8	18	72	98	73.5%
More adequate assessment	4	14	81	99	82%

TABLE II
Self-reported emotions during working in DBugIT

“I laughed out loud when I found the bug.”
”I really enjoyed the search for bugs and especially finding them.”
”The main feeling was excitement when I narrowed down the faulty behavior with a test case.”
”Disappointment since the bugs were too obvious.”
”Wanting to find more bugs then were placed in the assignment, so I guess it can really trigger you to want to test hard! Which is I guess good!”
”I was very curious to find the bugs, and I found the experience interesting and exciting.”
”I enjoyed testing.”
”I was eager to find the bug(s)”.
”To be honest, it was stressful for me because I knew there was a bug, but it wasn’t obvious.”
”A feeling of achievement when I found a bug.”
”Excitement when I figured out how the bug could be resolved.”
”I got a bit frustrated with some of the requirements, which were sometimes under-specified. ”
”Stress before, contentment after finding the bug.”
”Satisfaction and excitement when feeling like I found a complete answer/set of test cases or when I found the bug through a proper testing method.”

The results of the surveys (see Table I) show that 73.5 % of the participants found testing code with 100% bug-guarantee more exciting. An even higher percentage (82%) found this way of examination more adequate than a pen-and-paper or multiple choice exam. Table II shows that students experienced various emotions while hunting for bugs, including excitement and relief when they found the bug (something that we expected), but also fear and stress that they will not find the bug, frustration when the requirements were not clear, or disappointment that the bugs were too easy to find (which we did not anticipate).

Next, we used the self-driving car in a systems testing warming-up tutorial, where students in groups of 3 or 4 were encouraged to make mistakes and observe their effect. Although uncanny for non-engineering students, this tutorial

TABLE III
EVALUATIONS OF THE BUG-HUNTING IN EMBEDDED SOFTWARE.

Question	-	+-	+	Total	Agree
Testing is more exciting	7	21	83	111	75%
Liked the opportunity to make own mistakes	4	18	88	110	80%

a) b)

Fig. 4. a) A CS student hunting for bugs in embedded software; b) A bug-hunting workshop for IT alumni.

was very useful as it demonstrated that even a perfect software will fail if the wiring is wrong, or a component such as a LED, is broken. The students understood the importance of integration and system testing. The final systems testing assignment used the smart home. The task was to design a test plan for its fault-injected control software, execute this test plan, using all kinds of software testing techniques and diagnosis & visualization equipment, such as oscilloscopes, voltmeters, thermometers and luxmeters, and report on the bugs found, as can be seen in Fig. 4a). Each group of students received its own mutant. Examples of bugs we injected were: the garage door opens too late, the blue LED is correctly switched off, but the ventilator does not start, and the front door opens for *any* access code.

We deployed this hands-on approach in three consecutive offerings. The highest response rate we obtained the first time we deployed the intervention, back in 2022, when we used a non-anonymous survey that forced the students to respond. For scientific ethical reasons, we explained to students that a high participation rate was paramount to correctly evaluate the effect of our new initiative. The results (see Table III) show that 80% of the students liked the possibility to make their own mistakes. The vast majority of participants (75%) found that testing embedded code with intentionally injected bugs is more exciting than testing code written with the best intentions.

B. An introductory programming exam for non-CS students

In this course, taken by 244 students in economics and business, we replaced the pen-and-paper exam with an online exam in DBugIT. In this new exam, students had access to fault-seeded Python code snippets and their requirements. The task was (1) to find the bug, (2) to specify a test case that will find it, and (3) to suggest a bug fix. The exam consisted of four exercises and the final grade was calculated as an average of the four grades. Examples are a program that decides if a

TABLE IV
EXCERPT FROM ECONOMICS STUDENTS' EVALUATIONS.

Question	-	+-	+	Total	Agree
More adequate assessment	13	38	115	166	70%

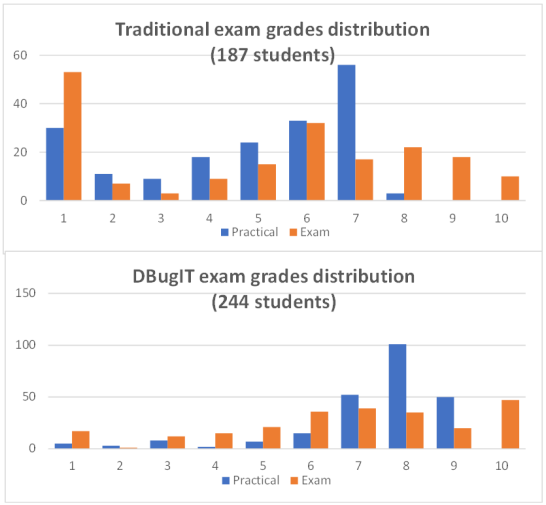


Fig. 5. The final grades (practical and exam) for the programming course before and after the intervention.

given year is a leap year or not, and a program that calculates the number of occurrences of a letter in a string.

In an anonymous survey, among other things, we asked students if they preferred an exam that tests code understanding and debugging skills over an exam asking them to just write code. The responses (see Table IV) show that 70% of the 166 participants answered positively to this question.

Fig.5 shows the grades obtained by the students involved in this experiment and the grades obtained by another cohort before the intervention. We notice that both have a Gaussian distribution, but with different means. Unfortunately, we could not properly compare students' performance, because in the traditional course the No-shows were also graded with a 1, while in our experiment we did not grade the No-shows. The only conclusion we can draw at this early stage is that a DBugIT exam generates similar grades with the pen-and-paper one.

C. A software testing refresher for IT alumni

The last experiment consisted of three workshops offered to alumni of a software engineering retraining program called MakeIT Work, organized by the Amsterdam University of Applied Sciences. In each workshop, around 30 students with little to no experience in testing attended a bug-hunting session in DBugIT, preceded by an introductory lecture. The students worked in groups of 4 to 6, and were guided by our teaching team in finding bugs hidden in assignments of increasing difficulty. In anonymous surveys, we asked students whether they found testing a 100% bug-guarantee code more exciting.

TABLE V
IT ALUMNI EVALUATIONS OF THE BUG-HUNTING WORKSHOP.

Question	-	+-	+	Total	Agree
Testing is more exciting	1	4	20	25	80%

The results of the student evaluations (see Table V) show that 80% of the 25 participants answered positively.

D. Discussion

We start by summarizing the answers to the research questions we outlined in the introduction.

RQ1. How does playing a bug-hunting game affect the excitement level of learning software testing? A high percentage of participants (73-80%) in both standalone- and embedded-software testing pilots found that bug-hunting added excitement to their learning.

RQ2. Is immersion in a bug-hunting experience a better way to assess software engineering skills compared to traditional exams? In all VU pilots, around 80% of the students found that an exam that tests debugging skills is a better assessment instrument compared to a traditional exam. The grades obtained in a programming exam conducted in DBugIT showed a similar Gaussian distribution.

RQ3. What emotions are triggered during a bug-hunting game? Software testers working in DBugIT reported experiencing emotions such as excitement, relief, fun, but also stress and frustration.

Although we can carefully conclude that our pilots were successful, we also need to mention the challenges we faced, such as the fabrication of bugs of the same level of difficulty and manually grading in a short time a large number of students. We also noticed that in all pilots, the students found more bugs than we have intended. This triggered very interesting discussions, like "What is a bug?", "What were the assumptions of the tester?", "How clear were the requirements?", etc.

A limitation of our approach is that we only used self-reports and leading open questions, whereas self-reports are known as a not very objective way to measure feelings. Moreover, it might be that some students wanted to please the teachers by answering positively and this might create an agreement bias in their answers. A mitigation measure will be to conduct more rigorous quantitative experiments, where we divide the students into two groups, a control one, with a traditional teaching and an experimental one, which will receive a gamified approach and compare their perception and exam performance. In order to keep the experiment fair and ethically correct we will offer in the end the gamified approach to the control group as well. Another mitigation measure will be to design qualitative studies to interview students about their feelings during bug-hunting. We will not participate in these interviews because we are the teacher and we want to prevent the power dynamics bias. Instead, we will assign a third independent person to conduct the interviews. We will also extend our research on using physiological sensors to monitor the emotions of bug-hunters in action [25]. We are aware though that this will be a challenging project from both a technical and ethical point of view.

IV. Related work

Traditional courses in software testing usually address only hypothetical standalone software [26], [27], [28]. Our approach is innovative because it uses authentic, executable code. The term "gamification" is rather young, but growing in popularity. Many educators used it to increase motivation, engagement, or students' outcomes in fields like music, mathematics, programming, software testing, or computer security [29], [30], [31], [32], [33], [34]. Compared to these initiatives, the games we developed are very simple, just sufficient to prove the concept. More gaming elements could be implemented in the future, such as badges, a bug jar, scoreboards, or a competition element. Innovative is also the use of miniature physical microcontroller-based devices. Although Arduino-based vehicles have been widely used to teach programming, software engineering, Internet of things and mobile robotics [35], we are not aware of cyber-physical systems like cars and smart homes used in software testing education. Using fault injection in teaching software testing is not new. It has been used for many years to assess the dependability of systems through seeding failure modes in hardware and software [36], [37], [38]. Mutation, just another word for faults injection, was used by many teachers in software testing courses [39], [40], [41], [42], [43], [44]. However, in all these cases, the emphasis was on finding bugs, and not so much on the quality of the test strategy. Also, using gamified mutation to assess skills in embedded software testing and programming courses is in our opinion rather new.

V. Conclusion and future work

We elaborated on an innovative idea to stimulate active and authentic learning in software engineering classes by using bug-hunting gamification. Three feasibility experiments with different groups of CS and non-CS students confirmed that the approach has the potential to make a change in the current situation by increasing the excitement level and assessment objectivity in programming and software testing learning. However, we are aware that more systematic research is needed. Future work includes adding more gamification elements, conducting more evaluation pilots to assess the transferability and generalization of the proposed approach, and continuing to promote fault injection in teaching both programming and testing in computing curricula.

VI. Acknowledgments

We would like to thank the DBugIT developers Marc Went, Robert Jansma, Viktor Bonev and Emil Apostolov; VU electronic and mechanical engineering groups for crafting our miniature cyber-physical systems; Hanneke Hoitink for her help in organizing the MakeIT Work pilots; The VU-BugZoo project and the MakeIT Work pilot were funded by the NRO, The Netherlands Initiative for Education Research, as part of a Comenius Teaching Fellow and a Kennisbenutting Plus grant.

REFERENCES

[1] R. P. Medeiros, G. L. Ramalho, and T. P. Falcão, "A systematic literature review on teaching and learning introductory programming in higher education," *IEEE Transactions on Education*, vol. 62, no. 2, pp. 77–90, 2019.

[2] A. Gomes and A. Mendes, "Learning to program - difficulties and solutions," in *International Conference on Engineering Education*, 01 2007, pp. 283–287.

[3] M. A. Foughali, "Some thoughts on teaching introductory programming and the first language dilemma (discussion paper)," in *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research*, ser. Koli Calling '23, 2024.

[4] I. Sommerville, *Software Engineering*. Addison-Wesley, 2010.

[5] V. Garousi, A. Rainer, P. Lauvås, and A. Arcuri, "Software-testing education: A systematic literature mapping," *Journal of Systems and Software*, vol. 165, p. 110570, 2020.

[6] L. F. Capretz, P. Waychal, J. Jia, D. Varona, and Y. Lizama, "Studies on the Software Testing Profession," in *Proceedings of the 41st International Conference on Software Engineering: Companion Proceedings*, 2019, p. 262–263.

[7] F. Cammaerts, P. Tramontana, A. C. R. Paiva, N. Flores, F. Pastor Ricós, and M. Snoeck, "Exploring students' opinion on software testing courses," in *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE '24, 2024, p. 570–579.

[8] T. Astigarraga, E. M. Dow, C. Lara, R. Prewitt, and M. R. Ward, "The emerging role of software testing in curricula," in *2010 IEEE Transforming Engineering Education: Creating Interdisciplinary Skills for Complex Global Environments*. IEEE, 2010, pp. 1–26.

[9] A. Deak, T. Stålhane, and G. Sindre, "Challenges and strategies for motivating software testing personnel," *Inf. Softw. Technol.*, p. 1–15, 2016.

[10] P. Waychal and L. F. Capretz, "Why a testing career is not the first choice of engineers," *ArXiv*, vol. abs/1612.00734, 2016.

[11] Y. Lizama, D. Varona, P. Waychal, and L. F. Capretz, *The Unpopularity of the Software Tester Role Among Software Practitioners: A Case Study*. Cham: Springer International Publishing, 2020, pp. 185–197.

[12] W. Diniz, M. Valença, C. França, A. Santos, and M. Pincovsky, "The skill gap in software industry: A mapping study," in *Brazilian Symposium on Software Engineering*, 2024.

[13] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, "From game design elements to gamefulness: defining" gamification"," in *Proceedings of the 15th international academic MindTrek conference: Envisioning future media environments*, 2011, pp. 9–15.

[14] R. N. Landers, "Developing a theory of gamified learning: Linking serious games and gamification of learning," *Simulation & gaming*, vol. 45, no. 6, pp. 752–768, 2014.

[15] N. Silvis-Cividjian, "Awesome bug manifesto: Teaching an engaging and inspiring course on software testing (position paper)," in *2021 Third International Workshop on Software Engineering Education for the Next Generation (SEENG)*, Jul. 2021, pp. 16–20.

[16] S. Papert and I. Harel, in *Constructionism*. Ablex Publishing Corporation, 1991.

[17] N. Silvis-Cividjian, R. Limburg, N. Althuisius, E. Apostolov, V. Bonev, R. Jansma, G. Visser, and M. Went, "VU-BugZoo: A Persuasive Platform for Teaching Software Testing," in *Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education*, ser. ITiCSE '20, 2020, p. 553.

[18] N. Silvis-Cividjian, M. Went, R. Jansma, V. Bonev, and E. Apostolov, "Good bug hunting: Inspiring and motivating software testing novices," in *ITiCSE '21*, ser. Annual Conference on Innovation and Technology in Computer Science Education, ITiCSE, Jun. 2021, pp. 171–177.

[19] Online, "Vue.js-The Progressive JavaScript Framework," https://vuejs.org/, accessed: 2020-10-10.

[20] E. M. Byrne, H. Jensen, B. S. Thomsen, and P. G. Ramchandani, "Educational interventions involving physical manipulatives for improving children's learning and development: A scoping review," *Review of Education*, vol. 11, no. 2, p. e3400, 2023.

[21] Pololu, "Pololu m3pi Robot with mbed Socket," https://www.pololu.com/product/2151, 2020, accessed: 2024-10-11.

[22] N. Silvis-Cividjian, G. Visser, J. Veltman, N. Althuisius, R. Limburg, and M. Molenaar, "House of the rising flames: A hands-on, bug-centered tutorial on embedded software testing," in *ITiCSE 2023*, ser. Annual Conference on Innovation and Technology in Computer Science Education, ITiCSE, Jun. 2023, pp. 581–582.

[23] NXP, "Freedom Development Platform for Kinetis® K66, K65, and K26 MCUs," https://www.nxp.com/design/design-center/development-boards-and-designs/general-purpose-mcus/freedom-development-platform-for-kinetis-k66-k65-and-k26-mcus:FRDM-K66F, 2024, accessed: 2024-10-11.

[24] B. Beizer, *Software Testing Techniques*. Van Nostrand Reinhold, 1990.

[25] N. Silvis-Cividjian, J. Kenyon, E. Nazarian, S. Sluis, and M. Gevonden, "On using physiological sensors and ai to monitor emotions in a bug-hunting game," in *ITiCSE 2024*, ser. Annual Conference on Innovation and Technology in Computer Science Education, ITiCSE, vol. 1, 2024, pp. 429–435.

[26] M. Pezzè and M. Young, *Software testing and analysis - process, principles and techniques.* Wiley, 2007.

[27] A. Mathur, *Foundations of Software Testing*. Addison-Wesley, 2008.

[28] P. C. Jorgensen, *Software Testing: a Craftsman's Approach*, 4th ed. Auerbach Publications, 2013.

[29] R. Nacional, "Gamifying education: Enhancing student engagement and motivation," vol. 5, pp. 716–729, 04 2023.

[30] R. Blanco, M. Trinidad, M. J. Suárez-Cabal, A. Calderón, M. Ruiz, and J. Tuya, "Can gamification help in software testing education? findings from an empirical study," *Journal of Systems and Software*, vol. 200, p. 111647, 2023.

[31] A. R. Fasolino and P. Tramontana, "Test smells learning by a gamification approach," in *Proceedings of the 3rd ACM International Workshop on Gamification in Software Development, Verification, and Validation*, ser. Gamify 2024, 2024, p. 30–33.

[32] K. Welbers, E. A. Konijn, C. Burgers, A. B. de Vaate, A. Eden, and B. C. Brugman, "Gamification as a tool for engaging student learning: A field experiment with a gamified app," *E-Learning and Digital Media*, vol. 16, no. 2, pp. 92–109, 2019.

[33] E. S. Rivera and C. L. P. Garden, "Gamification for student engagement: a framework," *Journal of Further and Higher Education*, vol. 45, no. 7, pp. 999–1012, 2021.

[34] C. Dichev and D. Dicheva, "Gamifying education: what is known, what is believed and what remains uncertain: a critical review," *International journal of educational technology in higher education*, vol. 14, pp. 1–36, 2017.

[35] J.-A. Marín-Marín, P. A. García-Tudela, and P. Duo-Terrón, "Computational thinking and programming with Arduino in education: A systematic review for secondary education," *Heliyon*, vol. 10, no. 8, p. e29177, 2024.

[36] J. Voas and G. McGraw, *Software Fault Injection*. John Wiley Sons, 1998.

[37] Mei-Chen Hsueh, T. K. Tsai, and R. K. Iyer, "Fault injection techniques and tools," *Computer*, vol. 30, no. 4, pp. 75–82, 1997.

[38] R. Natella, D. Cotroneo, and H. S. Madeira, "Assessing dependability with software fault injection: A survey," *ACM Comput. Surv.*, vol. 48, no. 3, Feb. 2016.

[39] S. Elbaum, S. Person, J. Dokulil, and M. Jorde, "Bug Hunt: Making Early Software Testing Lessons Engaging and Affordable," in *29th International Conference on Software Engineering (ICSE'07)*, 2007, pp. 688–697.

[40] B. S. Clegg, J. M. Rojas, and G. Fraser, "Teaching software testing concepts using a mutation testing game," ser. ICSE-SEET '17. IEEE Press, 2017, p. 33–36.

[41] K. Aaltonen, P. Ihantola, and O. Seppälä, "Mutation analysis vs. code coverage in automated assessment of students' testing skills," in *Proceedings of the ACM International Conference Companion on Object Oriented Programming Systems Languages and Applications Companion*, ser. OOPSLA '10, 2010, p. 153–160.

[42] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," *Computer*, vol. 11, no. 4, pp. 34–41, 1978.

[43] R. Smith, T. Tang, J. Warren, and S. Rixner, "An automated system for interactively learning software testing," in *Proceedings of the 2017 ACM Conference on Innovation and Technology in Computer Science Education*, ser. ITiCSE '17, New York, NY, USA, 2017, p. 98–103.

[44] G. Fraser, A. Gambi, M. Kreis, and J. M. Rojas, "Gamifying a software testing course with Code Defenders," in *Proc. of the ACM Technical Symposium on Computer Science Education (SIGCSE)*, ser. SIGCSE'19. ACM, 2019.